From Attack Behavior to Defense Failure: A Stage-Aware Framework for Evaluating APT Defense Capability

Anonymous Author(s)

Abstract

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → *Software creation and management*.

Keywords

APT defense evaluation, control-point attribution, stage-aware scoring, security benchmark

ACM Reference Format:

Anonymous Author(s). 2026. From Attack Behavior to Defense Failure: A Stage-Aware Framework for Evaluating APT Defense Capability. In *Proceedings of THE ACM WEB CONFERENCE 2026 (Conference acronym 'WWW).* ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Advanced Persistent Threats (APTs) have become a major threat to national, enterprise, and critical-infrastructure cybersecurity. According to M-Trends 2026, the global median dwell time rose to 14 days in 2025, while the median dwell time for cyber-espionage intrusions reached 122 days[4]. Verizon DBIR 2025 reports that credential abuse and vulnerability exploitation remain major initial attack vectors, and that the share of breaches involving third parties doubled to 30%[16]. IBM's 2024 Cost of a Data Breach report further shows that the global average cost of a data breach reached $4.88 million, and that about 70% of affected organizations experienced significant or moderate business disruption[5]. Taken together, these reports suggest that APT defense evaluation should focus not only on whether an intrusion is eventually found, but also on where defensive control is lost during a multi-stage campaign.

In response, organizations deploy layered defense stacks, including Endpoint Detection and Response (EDR), Next-Generation Firewalls (NGFW), Security Information and Event Management (SIEM), Privileged Access Management (PAM), and email security gateways. However, deployment does not imply effectiveness. A defense stack with more products is not necessarily more capable, and a system that fails to stop an entire APT campaign is not necessarily useless. What matters is whether the evaluation can show where the defense worked, where it failed, and which failures should be repaired first.

Conference acronym 'WWW, Dubai, United Arab Emirates

ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXXXX.XXXXXXX

Existing evaluation approaches provide useful pieces of this picture, but they do not directly produce repair-oriented, stage-aware defense capability scores. Attack knowledge bases such as MITRE ATT&CK provide a common language for describing attacker behavior, and ATT&CK Evaluations report product responses at the step level[11]. Vulnerability scoring and attack-graph methods analyze severity, reachability, or path risk[1, 8, 18]. These approaches are valuable, but they usually stop at attack techniques, product observations, vulnerabilities, or attack paths. They do not directly answer which victim-side control point first failed for a missed action, nor how that failure should affect a multi-stage APT defense score.

This paper starts from a different evaluation unit: an APT action paired with its observed defense result and its first-failed victim-side control point. The motivation is simple. The same ATT&CK technique may require different repairs in different contexts. A credential-related action, for example, may result from weak password policy, missing multi-factor authentication, exposed remote login, insufficient credential storage protection, or successful phishing. These cases may share an attack label, but they do not point to the same defensive repair. A useful scoring framework should therefore preserve the link between attacker behavior, defense observation, and repairable control failure.

Building such a framework raises three practical challenges. First, the attribution schema must be stable enough to handle heterogeneous APT actions while remaining useful for repair. Second, the attack-stage dimension must be represented separately from control-point domains, since the same type of defensive failure may appear at different phases of an attack. Third, action-level defense observations must be aggregated according to stage logic rather than by a simple average, while keeping score loss traceable after aggregation.

This paper proposes a stage-aware APT defense capability scoring framework based on control-point attribution. The framework first maps each in-scope attack action to the victim-side control point that failed first and is most relevant for repair. It then enriches the attributed action with a canonical attack phase, expected defense product categories, confidence information, and stage aggregation semantics. Finally, it aligns defense observations with benchmark actions, assigns action-level hit values, and aggregates them into playbook-level and multi-dimensional defense scores.

The main contributions of this paper are as follows:

- We propose a control-point attribution schema for APT defense evaluation, shifting the evaluation unit from "what the attacker did" to "which victim-side control failed first." The schema covers 9 top-level domains and 94 leaf-level control points, enabling action-level attribution of defensive failures.
- We design a benchmark construction process that connects control-point attribution with stage-aware scoring. It adds

canonical phase mapping, expected defense categories, confidence labels, and stage aggregation semantics to produce scoring-ready defense opportunity records.
- We develop a stage-aware scoring model that separates action-level hit values from stage importance and supports aggregation from actions to stages and playbooks. The model also reports scores by attack phase, control-point domain, and defense product category, making score loss easier to interpret.
- We evaluate the framework on 53 APT playbooks containing 1,758 attack actions. The results validate the mathematical behavior of the scoring engine and show that no single product category can independently cover the full APT attack surface, highlighting the need for coordinated defense-in-depth.

2 Related Work

Prior work on APT analysis and security evaluation provides several building blocks for this paper: standardized attack descriptions, step-level product observations, defensive technique knowledge bases, and vulnerability or path-risk models. These lines of work are useful, but they are organized around different evaluation units. Some describe attacker behavior, some report product responses, and others estimate vulnerability severity or attack reachability. This paper focuses on a different unit: an APT action whose defense result can be scored while remaining connected to a repairable victim-side control failure.

2.1 Attack Knowledge Bases and Step-Level Product Evaluation

MITRE ATT&CK provides a widely used language for describing adversary behavior. Its tactics, techniques, and procedures are used in threat intelligence, red-team planning, adversary emulation, and security product evaluation [9]. This makes ATT&CK a natural source for representing APT playbook actions, because it gives heterogeneous attack behaviors a shared vocabulary.

For defense scoring, however, an ATT&CK technique label is only a starting point. It describes what the attacker did, but it does not determine which victim-side control failed or which repair should be prioritized. A behavior involving credential use, remote execution, or command-and-control communication may be enabled by different defensive failures in different environments. Treating the ATT&CK label itself as the scoring unit would therefore make the result difficult to repair. Our framework keeps ATT&CK-style action descriptions, but adds a separate attribution layer for the first-failed victim-side control point.

MITRE ATT&CK Evaluations move closer to defense assessment by reporting how products behave during adversary-emulation steps [11]. This is valuable because it exposes product behavior at the level of concrete attack steps rather than only at the level of product features. In our setting, such step-level observations provide the raw evidence for defense scoring: an action may be prevented, blocked, detected, or missed.

The remaining issue is aggregation and attribution. A step-level matrix can show where a product responded, but it does not by itself decide which repairable control point should receive the score loss when an action is missed. It also does not directly express stage-aware value: blocking an initial-access path and detecting activity near exfiltration are both useful observations, but they should not contribute to an APT defense score in the same way. This paper therefore builds on step-level observations, but adds control-point attribution, canonical phase mapping, and stage aggregation semantics before computing scores.

2.2 Defensive Knowledge Bases and Control Capabilities

D3FEND complements attack knowledge bases by organizing defensive techniques and their relationships to adversary behaviors and digital artifacts [6, 10]. It is useful when the goal is to reason about which defensive techniques may apply to a given attack behavior. This capability-centered view is related to the defense-technology crosswalk used in our framework.

The scoring problem considered here is narrower. D3FEND can indicate that a class of defensive techniques may counter a behavior, but it does not decide which victim-side control point first failed in a particular playbook action. This distinction matters when score loss must be assigned to a repair target. For example, a missed credential-dumping action may involve endpoint monitoring, credential storage protection, privilege control, or response failure. Listing possible defensive techniques is useful, but the benchmark still needs a primary attribution for scoring.

Our control-point attribution schema and defense-technology crosswalk therefore use D3FEND-like defensive reasoning for a different purpose. The attribution schema assigns the action to the first-failed control point used in scoring. The crosswalk then links that control point to the product categories expected to prevent, block, or compensate for the failure. The goal is not to replace defensive knowledge bases, but to make their capability view usable in a repair-oriented scoring benchmark.

2.3 Vulnerability Scoring and Attack-Path Analysis

Another related line of work evaluates security through vulnerability severity, exploitability, reachability, and risk. CVSS provides a standardized way to describe and score software vulnerability severity [2, 8]. Attack-graph methods, including MulVAL and related work, model how vulnerabilities, privileges, and configurations can be combined into multi-step attack paths [12, 13]. Other studies assign quantitative or probabilistic metrics to attack graphs [3, 17, 18], and cyber-risk quantification estimates possible loss under different threat and control assumptions [1].

These methods make important security properties explicit. CVSS supports vulnerability prioritization. Attack graphs show how local weaknesses can combine into reachable attack paths. Risk models connect threats and controls to possible loss. These questions are related to APT defense evaluation, but they do not directly measure how a deployed defense stack responds to action-level APT behaviors.

The difference is most visible in the scoring unit. A high vulnerability score does not say whether a defense stack blocks initial access, detects execution, prevents lateral movement, or only observes activity after compromise. Similarly, an attack graph can

show that a path is reachable, but it does not directly convert action-level defense observations into phase-wise, control-domain, and product-category scores. Our framework uses a fixed set of APT playbook actions as the benchmark target and evaluates defense performance over those actions.

2.4 Position of This Work

The closest prior work therefore covers different parts of the evaluation pipeline. ATT&CK provides the attack language. ATT&CK Evaluations provide step-level product observations. D3FEND organizes defensive capabilities. CVSS and attack graphs reason about vulnerability severity, reachability, and risk. This paper connects these perspectives through a scoring layer that combines action-level defense results with control-point attribution and stage-aware aggregation.

This positioning follows directly from the limitations discussed in the introduction. If a score is computed only at the product or campaign level, it becomes difficult to explain where the loss came from. If all attack steps are treated equally, early interruption and late observation are flattened into the same type of contribution. If missed actions are not attributed to repairable control failures, the evaluation cannot guide remediation. The proposed framework addresses these issues by keeping score loss traceable to phases, actions, product categories, and victim-side control-point domains.

3 Background and Problem Statement

This section defines the evaluation setting considered in this paper. The goal is not to provide a general introduction to APTs, but to clarify the objects that are scored by our framework: APT playbook actions, defense observations, attack stages, and victim-side control-point failures.

3.1 APT Playbooks, Actions, and Stages

An APT campaign is usually described as a multi-stage process rather than as a single attack event. In this paper, we represent such a process using an APT playbook. A playbook contains a sequence of stages, and each stage contains one or more attack actions. An attack action is the smallest unit that can be aligned with a defense observation and later used for scoring.

This action-level view is important because a playbook-level result alone is too coarse. A defense system may fail to stop an entire campaign, but still detect or block important actions along the way. Conversely, a system may generate many alerts while still missing the action that allows the attacker to move to the next stage. For this reason, the benchmark should preserve action-level evidence before aggregating results to stages and playbooks.

Stages also differ in how their actions contribute to attacker progress. In some stages, several actions represent alternative ways to achieve the same objective; in others, multiple actions must be completed together; in still others, a group of related actions jointly describes an attacker activity such as discovery or collection. These differences mean that stage-level scoring should not treat every group of actions as a simple average. The exact aggregation rules are defined in Section 4.5; here, the key point is that stage structure is part of the evaluation problem.

3.2 Defense Observations and Evaluation Outcomes

A deployed defense stack may include endpoint, network, identity, email, logging, and response components. During an evaluation, these components may produce different kinds of evidence, such as prevention records, blocking events, alerts, logs, or analyst-facing detections. We refer to such evidence as defense observations.

A defense observation is not yet a score. It first needs to be aligned with a benchmark action. After alignment, the observation can be normalized into an action-level outcome. In this paper, we use four outcome levels: `PREVENTED`, `BLOCKED`, `DETECTED`, and `MISSED`. These levels distinguish actions that are removed by design, interrupted at runtime, observed by the defense, or not covered by available evidence.

This distinction is needed because different outcomes have different defensive meanings. Preventing an action through architecture or policy is not the same as detecting it after execution. Blocking an action during runtime is also different from merely logging it. A scoring framework should therefore preserve the outcome level before aggregating action results into larger scores.

3.3 From Attack Behavior to Control-Point Failure

Attack descriptions and defense repair targets are not the same object. An attack technique describes what the attacker did. A control-point failure describes why the victim environment allowed the action to succeed or why the defense did not respond effectively. For repair-oriented evaluation, the second question is essential.

The same attack behavior may correspond to different defensive failures in different contexts. For example, credential-related activity may be associated with weak password policy, missing multi-factor authentication, exposed remote login services, insufficient credential storage protection, phishing exposure, or delayed response. These cases may look similar from the attacker's side, but they imply different repair actions from the defender's side.

We therefore use the notion of a victim-side control point. A control point is a defensive mechanism, configuration, policy, monitoring capability, or operational process that can be evaluated and improved. For each in-scope action, the evaluation should identify the control point that failed first and is most relevant for repair. This attribution does not replace attack-phase information. Instead, it complements it: the control point explains where the defense failed, while the attack phase explains when the action occurs in the campaign.

3.4 Problem Statement

The problem studied in this paper can be stated as follows. Given a set of APT playbooks and action-level defense observations, construct a scoring framework that measures defense capability against multi-stage APT behaviors while keeping the resulting scores interpretable and repair-oriented.

More specifically, the framework should satisfy three requirements.

(1) **Repair-oriented attribution.** Each in-scope attack action should be connected to a victim-side control-point failure

that can guide remediation. Actions that do not correspond to a directly repairable victim-side failure should be retained for playbook completeness, but excluded from primary attribution-based scoring unless a concrete victim-side control failure can be identified.

(2) **Stage-aware aggregation.** Action-level outcomes should be aggregated according to the role of the action within its stage and the stage within the playbook. The framework should avoid treating all actions as equally valuable or all stages as having the same attacker-progress logic.

(3) **Traceable score reporting.** Aggregated scores should remain explainable after computation. A low score should be traceable to the relevant playbook, stage, action, defense outcome, product category, and victim-side control-point domain.

These requirements motivate the methodology in the next section. The proposed framework first attributes attack actions to repairable control points, then enriches them with phase and defense-category information, aligns defense observations with benchmark actions, and finally computes stage-aware and multi-dimensional scores.

4 Methodology

4.1 Overview

This section describes how the proposed method turns APT playbooks and defense observations into scores that remain traceable after aggregation. The central requirement is traceability: a score should not be interpreted only as an aggregate number, but should remain connected to the playbook, stage, attack action, and first-failed victim-side control point that contributed to it.

Fig. 1 gives the overall workflow. The workflow has three parts. First, benchmark construction converts raw playbook actions into scoring-ready records. Each action is assigned a control-point attribution, a canonical attack phase, expected defense categories, a confidence label, and stage aggregation semantics. These fields form the defense opportunity map used by the scoring engine.

Second, defense scoring aligns observed defense results with the benchmark records. A tested system reports action-level outcomes, which are normalized into four hit levels: `PREVENTED`, `BLOCKED`, `DETECTED`, and `MISSED`. These hit levels capture whether the action was removed by design, interrupted at runtime, only observed, or missed.

Third, the scoring model aggregates the aligned hit values. The aggregation is stage-aware: OR stages, AND stages, and Cluster stages use different rules because they represent different attacker progress conditions. The framework then reports not only an overall score, but also phase-wise scores, control-point domain scores, product-category scores, and action-level attribution details.

4.2 Defensive Weakness Attribution

The first step is to view each attack action from the defender's side. For every in-scope action, we identify the victim-side control point that failed first and is most relevant for repair. A control point may be a defensive mechanism, a configuration, a policy, or an operational process that can be evaluated and remediated independently. A successful attack action therefore indicates that some victim-side control was absent, misconfigured, bypassed, or insufficiently enforced.

This scope excludes actions that do not correspond to a directly repairable victim-side failure. Pre-compromise reconnaissance, attacker-side infrastructure preparation, and similar external prerequisite activities are retained for playbook completeness, but they are not included in primary weakness statistics or attribution-based scoring unless a concrete victim-side control failure can be identified.

This attribution step is needed because an ATT&CK label tells us what the attacker did, but not which defense failed. The same ATT&CK technique may be enabled by different weaknesses in different environments. For example, a credential-related action may result from weak password policy, missing multi-factor authentication, exposed remote login services, insufficient credential storage protection, or successful phishing. These cases may share similar attack labels, but they imply different repair actions.

We therefore interpret each action by asking a repair-oriented question: if only one control point could be fixed, which one would most directly prevent or disrupt this action? The answer is organized through a structured control-point schema. This schema is not organized as an attack sequence. It separates defensive failures along four design dimensions: security function, architectural location, defensive lifecycle phase, and trust source. These dimensions are not intended to form a strictly orthogonal category system. They are used as practical design constraints for separating control points that lead to different repair actions.

Security function follows the long-standing view that protection mechanisms should be separated by their security roles, such as authentication, authorization, privilege separation, execution control, and data protection [15]. Architectural location captures where the relevant trust boundary or enforcement point sits, which is consistent with zero-trust architecture's shift from static network perimeters toward users, assets, resources, and policy enforcement around them [14]. Defensive lifecycle phase distinguishes controls that prevent an action from succeeding from controls that detect, respond to, or recover from successful actions, aligning with the preventive–reactive distinction in cyber defense modeling [19]. Trust source captures whether the failed control is tied to externally supplied inputs, such as phishing messages, malicious documents, third-party collaboration, or supply-chain artifacts; this is related to attack-surface models that treat entry points, channels, and untrusted data items as key resources through which attackers interact with a system [7].

These dimensions also clarify why the nine domains are not arranged as an attack timeline. The domains describe different ways in which victim-side controls can fail, while the separate canonical phase layer captures when an action occurs in the attack chain. The schema contains nine top-level control-point domains and 94 fourth-level leaves. The domains indicate where score loss is concentrated, while the leaves identify the concrete controls that should be repaired.

Each in-scope action is assigned exactly one primary weakness leaf. This single-label rule keeps later scoring traceable. If one action had multiple primary attributions, the same failure could be counted several times, and score loss could not be linked to a clear remediation target. To keep attribution consistent, the schema uses

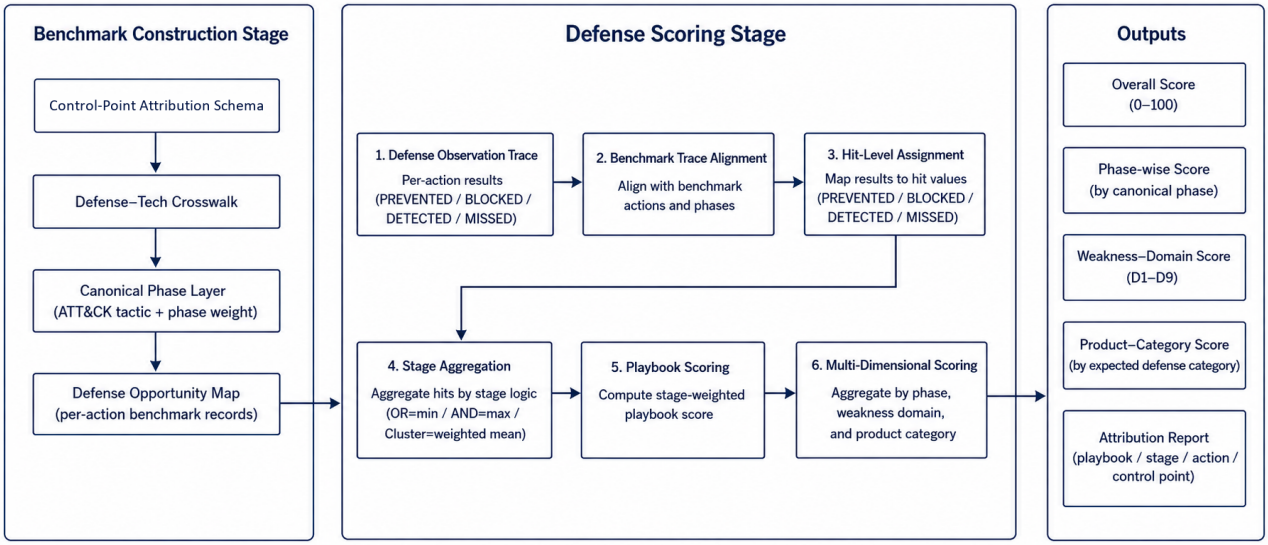

Figure 1: Workflow of the APT defense capability scoring framework.

Table 1: Top-level control-point domains.

Domain	Scope
D1	Identity, credential, and trust-root failures
D2	Authorization, isolation, and boundary-control failures
D3	Execution, host, and runtime-protection failures
D4	External exposure and remote-entry failures
D5	Communication and protocol-design failures
D6	Data protection, cryptographic-material, and recovery failures
D7	Observability, detection, and response failures
D8	External trust-input failures
D9	Availability and resilience failures

explicit boundary rules and the repair-oriented decision principle described above.

Some actions still involve more than one relevant weakness. Secondary weakness tags record such supporting conditions, including dual-control dependencies and entry–root-cause splits. These tags help explain attack chains, but they are not counted as primary attribution results and do not replace the primary leaf used for scoring.

After this step, raw attack actions have been converted into repair-oriented control-point failures. This attributed action set is the basis for canonical phase assignment, defense opportunity mapping, and stage-aware scoring.

4.3 Benchmark Construction

Control-point attribution identifies which victim-side control point failed, but it is not enough for scoring by itself. The scoring engine also needs to know when the action occurs in the attack chain, which defense product categories are expected to address it, and how the action should be aggregated with other actions in the same stage. Benchmark construction adds this information without changing the primary attribution result.

The process has three parts. Attributed actions are first assigned canonical attack phases using ATT&CK. Their control-point leaves are then linked to expected defense product categories. Finally, the enriched records are written into the defense opportunity map, which serves as the direct input to defense scoring.

4.3.1 Canonical Phase Assignment. The attribution domains describe where the victim-side defense failed; they deliberately do not encode when the action occurs in the attack chain. This temporal information is still needed for scoring and reporting. It allows the framework to distinguish defense performance across attack phases, apply phase-aware weights, and interpret stage-level results in relation to attacker progress. For example, interrupting an early action usually has more defensive value than observing a late action after the attacker has already progressed through several stages. We therefore add a separate canonical phase layer.

For each action, the framework assigns a canonical phase based on its ATT&CK technique. We use ATT&CK Enterprise tactics as the standard phase vocabulary. Some ATT&CK techniques can be associated with multiple tactics. We resolve this ambiguity using a predefined technique-to-phase mapping. Each technique is mapped to a single canonical phase before scoring, and this mapping remains fixed during defense scoring.

Stage-level phases are then derived from the action-level phases, for example by majority voting among the actions in the same playbook stage. This phase assignment is an additional benchmark field; it does not change the control-point attribution described in the previous subsection. The canonical phase field is later used by the scoring model to apply phase weights. The main intuition

is that earlier interruption has higher defensive value, while late-stage detection provides less benefit because the attacker has already progressed through much of the playbook.

4.3.2 Defense-Category Mapping. The primary weakness leaf identifies the failed control point. A scoring benchmark also needs to know which type of defense product should be expected to address that failure. For this purpose, we build a mapping from control-point leaves to defense product categories.

Each fourth-level leaf is mapped to two sets of defense categories. The first set is the primary defense category set. It contains product types that should directly prevent or block the corresponding weakness if deployed and configured correctly. The second set is the compensating defense category set. It contains product types that may detect, mitigate, or partially compensate for the weakness, but are not the most direct repair target.

For example, a brute-force action attributed to weak password and anti-brute-force policy is primarily related to identity and access management, while privileged access management or log monitoring may provide compensating support. Similarly, a C2 communication action attributed to missing egress proxy enforcement may be primarily related to network access control or next-generation firewall capabilities, while SIEM or IDS may provide secondary visibility.

This mapping allows later scoring to evaluate each product category within its expected coverage set. A product is therefore not judged against all benchmark actions equally, but against the actions that its category is expected to cover.

4.3.3 Defense Opportunity Map. The final output of benchmark construction is the defense opportunity map. Each record in the map corresponds to one attack action and combines the information needed for scoring: the playbook identifier, stage number, action description, ATT&CK technique, primary control-point leaf, top-level domain, canonical phase, expected defense categories, confidence label, and stage aggregation semantics.

The opportunity map keeps the primary attribution unique, but it can also record additional defense opportunities when they are available. In particular, secondary weakness tags can be expanded into additional leaves and defense categories. This preserves the repair-oriented primary attribution while still representing layered defense opportunities.

The opportunity map also records the aggregation semantics used for each playbook stage. These semantics specify how the action-level hit values within a stage should be combined during scoring. For example, a stage may represent alternative attack paths, sequentially required actions, or a loose cluster of related actions. The detailed aggregation rules are defined in the scoring model.

Table 2 summarizes the main fields used in the defense opportunity map.

Through these steps, benchmark construction turns attributed attack actions into structured defense opportunities. The resulting records connect four pieces of information required for scoring: what the attacker did, which victim-side control point first failed, which defense categories were expected to cover the action, and how the action should contribute to stage-level aggregation.

Table 2: Main fields in a defense opportunity record.

Field	Meaning
playbook, stage, action	Identifier of the attack action in the playbook
attack_id	ATT&CK technique or sub-technique identifier
primary_leaf	First-failed control-point leaf used for primary attribution
primary_domain	Top-level control-point domain of the primary leaf
canonical_phase	Standard attack phase used for phase-aware scoring
primary_defense	Product categories expected to directly address the weakness
compensating_defense	Product categories that may provide secondary detection or mitigation
confidence	Confidence of the primary attribution
stage_logic	Aggregation semantics used later at the stage level

4.4 Defense Result Alignment

Defense scoring starts by aligning the evaluated system's observations with the benchmark records. Each observation describes how the system responded to a specific playbook action. The alignment step matches the observation to the corresponding defense opportunity record and then converts the observed result into a normalized hit value.

We represent the defense observation trace as a set of action-level records. Each record contains the playbook identifier, the stage number, the action field used in the benchmark, and the observed result. Each defense observation is matched to the action record in the defense opportunity map with the same playbook identifier, stage number, and action field. After alignment, the observed result is associated with the action's control-point attribution, canonical phase, expected defense categories, confidence label, and stage aggregation semantics.

Actions without a matched defense observation are treated as missed actions. Under this rule, an action receives a positive hit value only when the evaluated defense system reports that it was prevented, blocked, or detected. Otherwise, the action is treated as `MISSED`. It also keeps the benchmark comparable across different defense systems: every system is evaluated against the same set of benchmark actions, and missing observations are interpreted as lack of demonstrated coverage.

Each matched observation is then converted into one of four hit levels. `PREVENTED` denotes architectural prevention, where the attack path is removed before the action can be attempted. `BLOCKED` denotes real-time interruption, where the action is attempted but stopped by the defense system. `DETECTED` denotes successful observation or alerting without stopping the action itself. `MISSED` denotes the absence of prevention, blocking, or detection. These levels are mapped to numerical hit values used by the scoring model.

The distinction between `PREVENTED` and `BLOCKED` is important. Prevention means that the attack path is unavailable by design, such as when an egress policy makes an unauthorized destination

Table 3: Defense observation results and hit values.

Result	Hit	Meaning
PREVENTED	1.0	The attack path is architecturally removed before the action can occur.
BLOCKED	0.9	The action is attempted but synchronously interrupted by the defense system.
DETECTED	0.5	The action is observed or alerted, but not stopped.
MISSED	0.0	The action is neither prevented, blocked, nor detected.

unreachable. Blocking means that the action is observed and interrupted during execution, such as when an endpoint product terminates a malicious process. Both are stronger than detection-only outcomes, but prevention is assigned a slightly higher hit value because it does not depend on runtime recognition of the specific attack instance.

This alignment step produces a unified action-level scoring input. Each benchmark action is associated with both an expected defense opportunity and an observed defense result.

4.5 Stage-Aware Scoring Model

After defense result alignment, each benchmark action has an action-level hit value and a set of benchmark metadata, including its canonical phase, attribution confidence, and stage aggregation semantics. The scoring model uses these fields to compute defense scores at three levels: action, stage, and playbook.

The main design principle is to separate defense quality from stage importance. The stage score measures how well the defense performed within a stage, while the stage weight determines how much that stage contributes to the playbook-level score. Let a denote an attack action. Its hit value $h(a) \in [0, 1]$ is obtained from the aligned defense result described in the previous subsection.

The action weight $w(a)$ is defined as

$$w(a) = W_{\text{phase}}(\phi(a)) \cdot C_{\text{conf}}(\gamma(a)),$$

where $\phi(a)$ is the canonical phase assigned to action a, W_{phase} is the phase-weight function, $\gamma(a)$ is the attribution confidence label, and C_{conf} is the confidence factor. In our benchmark, the confidence factor reduces the contribution of actions whose attribution is less certain. For example, high-, medium-, and low-confidence actions can be assigned factors of 1.0, 0.8, and 0.5, respectively.

For a playbook stage s, let A_s be the set of actions in that stage. The stage score S_s is computed according to the aggregation semantics of the stage. We use three aggregation types: OR, AND, and Cluster.

For an OR stage, the attacker only needs one action in the stage to succeed in order to proceed. Therefore, the defense must block all actions in the stage. The stage score is defined as

$$S_s = \min_{a \in A_s} h(a).$$

This reflects the weakest-link constraint: if any action is missed, the stage is considered poorly defended.

For an AND stage, the attacker needs all actions in the stage to succeed in order to complete the stage objective. Therefore, blocking any one required action can disrupt the stage. The stage score is defined as

$$S_s = \max_{a \in A_s} h(a).$$

This reflects the strongest defensive interruption within the stage.

For a Cluster stage, the actions do not have a strong logical dependency. They are treated as a group of related behaviors rather than as strict alternatives or strict requirements. The stage score is computed as a weighted average:

$$S_s = \frac{\sum_{a \in A_s} h(a) w(a)}{\sum_{a \in A_s} w(a)}.$$

In this case, actions with higher phase importance or higher attribution confidence contribute more to the stage score.

The stage weight is computed separately from the stage score:

$$W_s = \sum_{a \in A_s} w(a).$$

This separation avoids mixing the quality of defense within a stage with the importance of that stage in the overall playbook. In particular, OR and AND stage scores are not weighted internally, because their semantics are determined by the attacker's path structure. The weights are used when aggregating stages into a playbook score.

For a playbook P with stages $S(P)$, the playbook-level score is

$$\text{Score}(P) = \frac{\sum_{s \in S(P)} W_s S_s}{\sum_{s \in S(P)} W_s}.$$

The overall defense capability score is then computed by aggregating the scores over all evaluated playbooks:

$$\text{Score}_{\text{overall}} = \frac{1}{|\mathcal{P}|} \sum_{P \in \mathcal{P}} \text{Score}(P),$$

where $\mathcal{P}$ denotes the set of playbooks in the benchmark.

This formulation keeps the final score traceable: a score loss can be traced back from the overall score to a playbook, then to a stage, and finally to the individual actions and control-point failures that caused the loss.

4.6 Multi-Dimensional Score Outputs

The overall score is useful, but it is only a starting point. Two defense systems may receive similar aggregate scores for different reasons: one may detect many actions late, while another may cover fewer actions but block them reliably within its intended scope. For this reason, the framework reports scores from multiple perspectives: overall performance, attack phase, control-point domain, defense product category, and action-level attribution.

The phase-wise score groups results by canonical attack phase. This view answers the question: at which attack phases does the defense system lose the most score? Since the scoring model assigns higher value to earlier interception, phase-wise results help distinguish early-stage blocking from late-stage observation. This directly supports stage-aware analysis of APT defense capability.

The control-point domain score groups results by the primary victim-side control-point domain. This view answers a different

question: which type of defensive control fails most often or contributes most to score loss? For example, a low score in an identity-related domain points to weaknesses in credential and trust controls, while a low score in an observability-related domain indicates insufficient detection or response. This decomposition turns an aggregate score into repair-oriented feedback.

The product category score evaluates performance within the expected coverage of each defense product category. For a product category c, let A_c denote the set of actions whose defense opportunity records include c as a primary or compensating defense category. The coverage of c is the fraction of benchmark actions that fall into this expected coverage set:

$$\text{Coverage}(c) = \frac{|A_c|}{|\mathscr{A}|},$$

where $\mathscr{A}$ is the set of benchmark actions.

The hit rate of c is computed only within its coverage set:

$$\text{HitRate}(c) = \frac{\sum_{a \in A_c} \tilde{h}_c(a)}{|A_c|}.$$

Here, $\tilde{h}_c(a)$ is the hit value credited to category c. Primary defense matches receive full hit credit, while compensating defense matches may receive discounted credit because they detect or mitigate a weakness but do not directly eliminate the failed control point.

Reporting both coverage and hit rate is important. Coverage describes how much of the benchmark a product category is expected to address. Hit rate describes how well it performs within that expected scope. A product with narrow coverage but high hit rate has a different defense profile from a product with broad coverage but low hit rate, even if their aggregate scores are similar.

Finally, the framework produces an action-level attribution report. For each score loss, the report retains the playbook, stage, action, ATT&CK technique, canonical phase, primary control-point leaf, defense categories, and observed result. This report provides the trace from aggregate score back to concrete attack actions and failed control points. As a result, the output can support both high-level comparison and detailed remediation analysis.

4.7 Prototype Method Validation

The preceding subsections define the attribution schema, benchmark construction process, result alignment procedure, scoring model, and multi-dimensional outputs. Before applying the framework to real defense-system replay experiments, we perform a set of prototype validation experiments on the benchmark and scoring infrastructure. These experiments test whether the proposed methodological components are reproducible, whether the control-point–to–defense mapping is reasonable, and whether the scoring engine behaves correctly under boundary and parameter-variation cases.

First, we evaluate the reproducibility of defensive weakness attribution through a pilot inter-rater reliability study. A stratified sample of 50 ATT&CK techniques and sub-techniques was independently labeled by two annotators using the same coding guide.

Table 4: Prototype validation of defensive weakness attribution.

Field	Agreement	κ	Interpretation
Scope	100.0%	1.000	Almost perfect
L1 domain	96.0%	0.953	Almost perfect
L4 leaf	86.0%	0.856	Almost perfect
Confidence	82.0%	0.679	Substantial
Mapping method	82.0%	0.687	Substantial
Secondary tag	66.0%	0.173	Slight

Table 5: Prototype validation of defense-category mapping.

Validation item	Result
External agreement with native control labels	74.8% (1,297 / 1,733 actions)
Highest category agreement	DLP 96.4%, NET 93.6%, EGW 90.3%
Endpoint-category agreement	AV 73.5%, EDR 71.9%
Primary defense internal agreement	97.1% (34 / 35 leaves), κ = 0.485
Compensating defense internal agreement	45.7% (16 / 35 leaves), κ = −0.094

Agreement was measured at multiple levels, including scope decision, top-level domain, leaf-level attribution, confidence label, mapping method, and secondary weakness tag. Table 4 summarizes the results.

The result shows that the primary attribution levels are reproducible in the pilot setting. The high agreement at the L1 and L4 levels indicates that the main control-point attribution can be applied consistently. In contrast, the secondary tag has much lower agreement, so it is retained as qualitative chain-context information rather than as a primary quantitative scoring field.

Second, we validate the defense-category mapping used by the Defense–Tech Crosswalk. The external validity check compares the crosswalk categories with the dataset's native control labels. Among 1,733 comparable actions, the two sources agree on 1,297 actions, giving an agreement rate of 74.8%. We also perform an internal consistency check in which an independent annotator maps a stratified sample of 35 leaves to the 29 defense categories without seeing the existing crosswalk. Table 5 summarizes these checks.

These results suggest that the primary defense mapping is stable enough to support product-category scoring. The lower consistency for compensating defenses indicates that compensating coverage is less sharply defined than primary coverage. Therefore, compensating matches are treated as partial credit and should be interpreted more cautiously.

Third, we check the mathematical behavior of the scoring model using synthetic coverage traces. These traces are not real defense experiments; they are boundary and coverage simulations. A perfect-defense trace assigns `PREVENTED` to every benchmark action, and

Table 6: Prototype scoring checks with synthetic coverage traces.

Simulation	Configuration	Score
Perfect	All actions are PREVENTED	100.00
None	All actions are MISSED	0.00
Ideal EDR	EDR primary coverage → BLOCKED	44.40
Ideal NGFW	NGFW/NET primary coverage → BLOCKED	24.93

Table 7: Sensitivity and aggregation robustness checks.

Check	Result
Perfect / none under all parameter settings	100 / 0 in all configurations
EDR score range	43.41–46.67
NGFW score range	23.62–27.57
EDR/NGFW ratio range	1.57–1.94
Discovery-dominated Cluster alternative	Maximum score delta 1.07
Action-count weighted playbook aggregation	Maximum score delta 0.78

a no-defense trace assigns MISSED to every action. Two idealized product-category traces simulate systems that block all actions in their expected EDR or NGFW/NET primary coverage. Table 6 reports the resulting scores.

The perfect and none traces verify the expected upper and lower bounds of the scoring engine. The ideal EDR and ideal NGFW simulations show that a single product category cannot cover the whole benchmark even when it performs perfectly within its expected scope. This result reflects the multi-stage and multi-control nature of APT defense rather than the performance of a specific commercial product.

Finally, we test the robustness of the scoring results under reasonable parameter and aggregation variations. We vary phase weights, confidence factors, and compensating-defense discounts, and we also compare alternative aggregation choices for discovery-heavy Cluster stages and playbook-level aggregation. Table 7 summarizes the main results.

The sensitivity analysis shows that the boundary behavior of the scoring model remains stable across parameter settings, and the relative ordering between the idealized EDR and NGFW traces does not change. The aggregation robustness checks also show limited score variation under alternative but reasonable aggregation choices. These results support the use of the proposed scoring model as a stable prototype benchmark infrastructure. The later evaluation section will use real defense-system observations to test how deployed defenses perform under this methodology.

5 Evaluation

6 Discussion

7 Conclusion

References

[1] Arnau Erola, Ioannis Agrafiotis, Jason R. C. Nurse, Louise Axon, Michael Goldsmith, and Sadie Creese. 2022. A System to Calculate Cyber Value-at-Risk. *Computers & Security* 113 (2022), 102545. doi:10.1016/j.cose.2021.102545

[2] FIRST. 2019. Common Vulnerability Scoring System Version 3.1: Specification Document. https://www.first.org/cvss/v3.1/specification-document. Accessed: 2026-05-26.

[3] Marc Frigault, Lingyu Wang, Anoop Singhal, and Sushil Jajodia. 2008. Measuring Network Security Using Dynamic Bayesian Network. In *Proceedings of the 4th ACM Workshop on Quality of Protection*. ACM, 23–30. doi:10.1145/1456362.1456368

[4] Google Cloud Mandiant. 2026. M-Trends 2026: Data, Insights, and Strategies From Mandiant Frontline Investigations. https://cloud.google.com/blog/topics/threat-intelligence/m-trends-2026. Accessed: 2026-05-26.

[5] IBM Security. 2024. Cost of a Data Breach Report 2024. https://www.ibm.com/think/insights/whats-new-2024-cost-of-a-data-breach-report. Accessed: 2026-05-26.

[6] Peter E. Kaloroumakis and Michael J. Smith. 2021. Toward a Knowledge Graph of Cybersecurity Countermeasures. In *Proceedings of the 2021 Workshop on Cyber Security Experimentation and Test (CSET)*. https://d3fend.mitre.org/resources/D3FEND.pdf

[7] Pratyusa K. Manadhata and Jeannette M. Wing. 2011. An Attack Surface Metric. *IEEE Transactions on Software Engineering* 37, 3 (2011), 371–386. doi:10.1109/TSE.2010.60

[8] Peter Mell, Karen Scarfone, and Sasha Romanosky. 2006. Common Vulnerability Scoring System. *IEEE Security & Privacy* 4, 6 (2006), 85–89. doi:10.1109/MSP.2006.145

[9] MITRE. 2026. MITRE ATT&CK. https://attack.mitre.org/. Accessed: 2026-05-26.

[10] MITRE. 2026. MITRE D3FEND: A Knowledge Graph of Cybersecurity Countermeasures. https://d3fend.mitre.org/. Accessed: 2026-05-26.

[11] MITRE Engenuity. 2026. ATT&CK Evaluations: Methodology Overview. https://evals.mitre.org/methodology-overview/. Accessed: 2026-05-26.

[12] Xinming Ou, Wayne F. Boyer, and Miles A. McQueen. 2006. A Scalable Approach to Attack Graph Generation. In *Proceedings of the 13th ACM Conference on Computer and Communications Security*. ACM, 336–345. doi:10.1145/1180405.1180446

[13] Xinming Ou, Sudhakar Govindavajhala, and Andrew W. Appel. 2005. MulVAL: A Logic-based Network Security Analyzer. In *Proceedings of the 14th USENIX Security Symposium*. USENIX Association. https://www.usenix.org/conference/14th-usenix-security-symposium/mulval-logic-based-network-security-analyzer

[14] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. 2020. *Zero Trust Architecture*. NIST Special Publication 800-207. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-207

[15] Jerome H. Saltzer and Michael D. Schroeder. 1975. The Protection of Information in Computer Systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. doi:10.1109/PROC.1975.9939

[16] Verizon Business. 2025. 2025 Data Breach Investigations Report. https://www.verizon.com/business/resources/reports/2025-dbir-data-breach-investigations-report.pdf. Accessed: 2026-05-26.

[17] Lingyu Wang, Tania Islam, Tao Long, Anoop Singhal, and Sushil Jajodia. 2008. An Attack Graph-Based Probabilistic Security Metric. In *Data and Applications Security XXII (Lecture Notes in Computer Science, Vol. 5094)*. Springer, 283–296. doi:10.1007/978-3-540-70567-3_22

[18] Lingyu Wang, Anoop Singhal, and Sushil Jajodia. 2007. Measuring the Overall Security of Network Configurations Using Attack Graphs. In *Data and Applications Security XXI (Lecture Notes in Computer Science, Vol. 4602)*. Springer, 98–112. doi:10.1007/978-3-540-73538-0_9

[19] Ren Zheng, Wenlian Lu, and Shouhuai Xu. 2018. Preventive and Reactive Cyber Defense Dynamics Is Globally Stable. *IEEE Transactions on Network Science and Engineering* 5, 2 (2018), 156–170. doi:10.1109/TNSE.2017.2736567

A Appendix

Received 7 October 2025; revised 24 December 2025; accepted 13 January 2026